

CuCM: a GPU-Powered Context-Mixing Compressor for Archival Storage

Abstract

The explosive growth of global data has created an increasing demand for archival storage, where efficient compression is crucial to reduce capacity cost. However, existing archival compressors face a fundamental tradeoff: mainstream methods (e.g., ZSTD with level 21/22) offer limited compression ratios, while context-mixing compressors (e.g., LPAQ) achieve higher ratios but are often too slow for practical use.

We present CuCM, a GPU-powered context-mixing compressor that overcomes this tradeoff. By introducing pre-learning and batch update mechanisms, CuCM effectively resolves the inherent data dependencies in the autoregressive modeling steps of context-mixing compressors. To the best of our knowledge, this is the first design that enables a context-mixing compressor to run at a high degree of parallelism in GPU architectures, achieving high throughput and a strong compression ratio. Experimental results show that CuCM achieves up to $12.6\times$ higher throughput than LPAQ while maintaining comparable compression ratios across both general-purpose and archival datasets.

1 Introduction

The global data volume is growing at an unprecedented pace. According to IDC’s recent white paper [1], the global datasphere reached 159 ZB (1 ZB = 1,073,741,824 TB) in 2024 and is projected to surpass 384 ZB by 2028, with its growth rate continuing to accelerate. This explosive expansion of digital content has led to a massive demand for archival storage, which in turn imposes tremendous pressure on storage capacity and cost [2]. To address this challenge, data compression has become one of the most fundamental and effective techniques for mitigating storage overhead in large-scale archival systems.

In archival scenarios, the design priorities of compression algorithms differ significantly from those in real-time or transactional storage [3, 4, 5]. Archival compressors typically prioritize high compression ratios, as they directly determine long-term storage efficiency and cost-effectiveness. Conversely, compression speed is relatively less critical, since archival data can be compressed offline over extended periods and is rarely accessed once stored. This tradeoff allows more computationally intensive algorithms to be adopted if they can achieve substantially better compression efficiency.

Existing archival compressors can be broadly classified into two categories: (1) *Transform-based compressors*, such as LZMA [6] and ZSTD [7], which combine techniques (such as LZ77 [8], Huffman coding, and range coding) to reduce redundancy by transforming data representation. For example, LZ77 replaces recurring sequences with short references, while Huffman coding assigns shorter codes to more frequent symbols. (2) *Context-mixing compressors*, such as the PAQ family [9], which employ advanced probabilistic modeling to predict the likelihood of upcoming symbols based on rich contextual information, achieving superior compression ratios through fine-grained adaptive modeling [10, 11, 12].

However, both categories face intrinsic limitations. (1) Transform-based compressors, though fast and practical, often fail to fully capture the deep structural regularities within data, resulting in suboptimal compression ratios. It is a critical weakness in archival scenarios where every percentage point in space saving matters. (2) Context-mixing compressors can achieve state-of-the-art compression ratios, but their autoregressive modeling process is inherently sequential and computationally expensive. Each prediction depends on the previous symbol and model update, making parallel execution extremely challenging. As a result, context-mixing methods often exhibit orders-of-magnitude lower throughput, rendering them impractical for archival workloads.

This dilemma highlights a pressing need for a new archival compressor that can simultaneously deliver high compression ratios and practical performance.

To this end, we propose CuCM, a GPU-powered context-mixing compressor that overcomes this tradeoff. ❶ CuCM introduces a prelearning mechanism to enable chunk-level parallelism during compression. It captures the statistical distributions and semantic patterns from sampled data and shares this knowledge across chunks, thereby eliminating inter-chunk dependencies while preserving essential contextual information. ❷ CuCM introduces a batch-updating mechanism in modeling to enable bit-level parallelism during compression. It strategically keeps the model fixed within each processing batch and performs a controlled batch update after compression, ensuring that parallel predictions remain deterministic and fully consistent with sequential decoding. ❸ Supported by these techniques, CuCM effectively decouples sequential dependencies in context mixing at multiple granularities, enabling it to fully leverage the massive arithmetic throughput and parallelism of modern GPUs.

Experiment results suggest that, CuCM achieves up to $12.6\times$ higher throughput than existing SOTA archival compressor, while maintaining comparable compression ratios across both general-purpose and archival datasets.

2 Backgrounds and Motivations

2.1 Context-Mixing Compression

Context-mixing compression is a technique that achieves high compression ratios by accurately capturing the statistical characteristics of the input data. A typical context-mixing compressor employs multiple bit-level prediction models to internalize the data’s generative regularities. Accordingly, these models produce well-calibrated probabilities that lower cross-entropy, enabling shorter codes and hence better compression. Owing to its fine-grained probabilistic modeling, context mixing often achieves the highest compression ratios among lossless compressors. This makes it particularly attractive for archival storage, where space efficiency outweighs compression speed, and even small improvements in ratio can translate into significant long-term cost savings.

However, context mixing is notoriously inefficient [13, 14]. It involves ❶ **intensive computations** for probabilistic modeling and suffers from ❷ **strong sequential dependency** that prevents effective parallelization. Its prediction models are adaptively updated after every processed bit, and each prediction depends on all previously encoded bits. As illustrated in the CPU path of Fig. 1, during compression, the com-

pressor processes one bit at a time: it generates a context from prior bits, produces model predictions, combines them through a context-dependent mixer, and feeds the final prediction into an entropy coder such as arithmetic coding. Once the actual bit value is revealed, all models are immediately updated to refine their internal statistics before moving to the next bit. Decompression mirrors this process, with the entropy coder decoding each bit based on the evolving predictions of the same models.

Several prior studies [13, 14] have attempted to accelerate the fast context-mixing compressor LPAQ [9] using FPGA platforms. However, such dedicated hardware is both costly and inflexible. For example, a Xilinx Virtex UltraScale+ XCVU9P card costs over \$61,000 and cannot serve general purpose tasks. Due to the prohibitive cost and limited accessibility of these devices, these prior works were evaluated only in simulation rather than on real hardware.

2.2 Accelerating Context-Mixing Compression with General-Purpose GPUs

Modern general-purpose GPUs offer two decisive advantages over CPUs: substantially higher arithmetic throughput and massive thread-level parallelism. For instance, an NVIDIA L40S GPU delivers a theoretical peak of approximately 91.6 TFLOPS [15], compared to 2.15 TFLOPS for an Intel Xeon Platinum 8468V processor [16].

Such exceptional computational throughput makes GPUs a promising platform for accelerating context-mixing compression. GPUs provide the raw computational power necessary to substantially reduce the modeling latency that limits CPU-based implementations (the first inefficiency in § 2.1). Their ability to sustain high-rate floating- and fixed-point operations can effectively handle the frequent probability estimation, parameter updating, and prediction steps that dominate the modeling cost in context mixing.

However, the strong autoregressive dependency in context mixing (the second inefficiency in § 2.1) undermines this acceleration potential, as it constrains parallel execution and prevents full utilization of GPU resources. To unlock the potential of GPU acceleration, there requires a new approach that relaxes these dependencies and exposes fine-grained parallelism, thereby fully exploiting GPU compute capacity to improve end-to-end throughput without compromising compression ratio.

3 Design and Implementation

3.1 Overall Framework

To unlock the acceleration potential of GPUs for context-mixing compression, we design CuCM. CuCM rearchitects the fast context-mixing compressor, LPAQ [9], and exposes both chunk-level and bit-level parallelism for efficient execution on GPU. As shown in Fig. 1, the overall framework of CuCM has four steps:

(1) **Chunking**: Apart from the part left for prelearning, the input data is divided into independent chunks that can be processed in parallel. Each chunk is assigned to a fixed number of GPU threads. When the need of memory of chunks exceeds GPU capacity, these chunks are divided into different rounds.

(2) **Initialization and Prelearning**: The compressor initializes all models and required data structures in GPU memory. During prelearning, a small subset of the input data is processed first to build initial context models, which serve as a shared

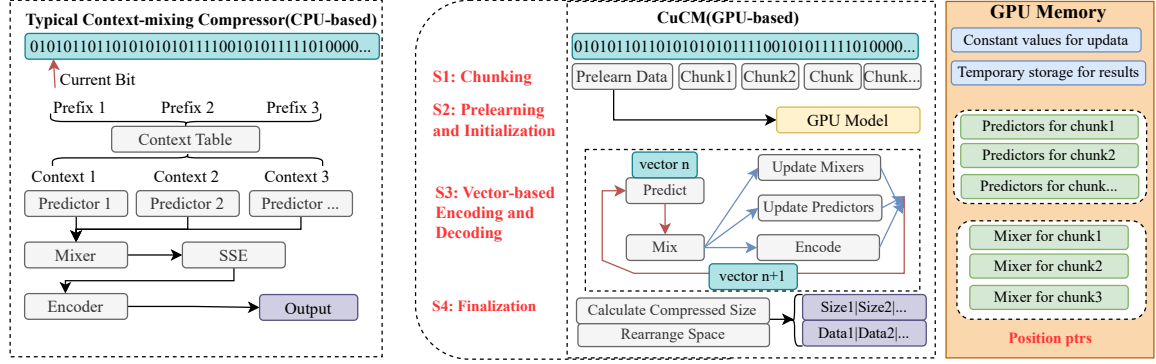


Figure 1: CuCM’s framework compared to CPU Context Mixing Compressor.

foundation for subsequent chunks to better exploit cross-chunk correlations.

(3) **Vector-based Encoding and Decoding:** Each chunk is further divided into vectors containing multiple bits. Threads process these vectors sequentially but operate on bits within a vector in parallel. During compression, bits are predicted and compressed concurrently. During decompression, the GPU speculatively predicts several bits in advance to enable parallel decoding. Model updates occur after each complete vector is processed.

(4) **Finalization:** The compressor flushes the remaining entropy-coded data, calculates the length of each chunk, writes the output, releases GPU memory, and terminates the process.

3.2 Prediction models used in CuCM

For each chunk, CuCM employs eight prediction models and one mixer to predict next bit. The k -th prediction model takes the first k bytes before the current bit and the current byte as the context, with k ranging from 0 to 7. These bytes are hashed into a context value within $1u \ll 25$ as the prediction input for predictors using rolling hash. The mixer is just a simple logistic mixer that takes the predictions from all predictors as input and outputs a final prediction by calculating a weighted average of the inputs. Thus, one chunk takes about $32\text{MB} \times 8 = 256\text{MB}$ of memory in the GPU memory to store its parameters. The benefit of using such a large hash table is that it reduces hash collisions and thus ensures prediction accuracy to combat the loss of compression ratio caused by chunking.

3.3 Parallelism in Compression

Compression parallelism in CuCM is primarily realized at both the chunk and bit levels, enabled by two key techniques: ❶ prelearning, which shares context across chunks; and ❷ batch updating, which keeps the model fixed within each vector to allow safe intra-vector parallelism.

(1) Chunk-level Parallelism. At the chunk level, CuCM distributes independent data chunks across GPU thread blocks for concurrent processing. Each chunk maintains its own set of predictors and mixers, and all chunks can be compressed in parallel, as their model states are isolated.

To maintain compression quality while ensuring such independence, CuCM em-

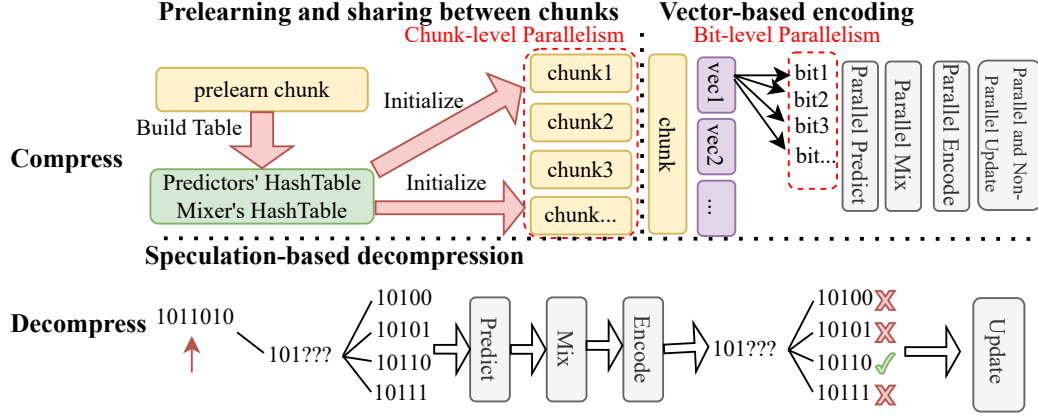


Figure 2: CuCM achieves parallelism by chunk-level and bit-level when compressing data and uses a speculation-based approach when decompressing data.

plays a prelearning phase that builds a shared initial model before GPU compression begins. This model is trained on a small portion of input data and then copied to all chunks in GPU memory, allowing each chunk to start from a well-calibrated context state rather than a cold start. As a result, the prelearned model provides cross-chunk contextual consistency without introducing runtime dependencies among chunks.

In practice, CuCM employs several CUDA kernels corresponding to the major operations in context-mixing compression: prediction, mixing, encoding, and parameter updating (for both predictors and mixers). Predictors record probability states of contexts to estimate the likelihood of the next bit being 1, while the mixer combines predictions from all predictors through a logistic function that computes a weighted average to produce the final probability.

For each vector within a chunk, CuCM sequentially launches the prediction and mixing CUDA kernels, passing the predicted probabilities between them. Subsequent stages such as encoding and updating can be executed concurrently across different chunks, as they are independent.

We assign one thread per chunk to update predictors to ensure deterministic results during compression. This is necessary because a predictor’s state update depends not only on the current bit and context, but also on its entire sequence of previous updates. Mixers, however, are updated by multiple threads in parallel, as their updates simply involve additive weight adjustments. These concurrent updates are safely managed using CUDA’s atomic operations.

(2) Bit-level Parallelism. At the bit level, CuCM achieves fine-grained concurrency within each chunk through vector-based encoding. Each chunk is divided into vectors containing multiple bits, and GPU threads process these vectors sequentially but handle all bits within a vector in parallel.

To enable such parallelism without breaking model consistency, CuCM adopts a batch-updating mechanism within each vector. During compression, the model parameters are kept constant while all bits in a vector are being predicted and encoded in parallel. Once an entire vector is processed, model updates (the context

within a vector) are applied in batch. This design ensures that parallel predictions remain deterministic and consistent with sequential decoding, while also minimizing synchronization overhead.

To maximize warp utilization, the vector size is enforced as a multiple of 32 so that all threads of each warp can process the exact same vector and no idle threads are left except for the last few bits. During kernel launches, CuCM arranges threads and memory layouts to ensure coalesced global-memory access. Specifically, predictions generated by the prediction kernel are stored so that threads accessing the same predictor across different bits of a vector read from contiguous memory locations. For example, the prediction corresponding to the i -th chunk, j -th bit of a vector, and k -th predictor is stored at $i \times \text{vector_size} \times \text{predictors_num} + k \times \text{predictors_num} + j$, allowing warp threads to access continuous addresses when updating the same predictor across bits, thereby minimizing memory divergence.

Through batch updating strategy, CuCM enables concurrent prediction, mixing, and encoding at the bit level while preserving the sequential semantics required for correct decompression.

3.4 Parallelism in Decompression

Decompression poses greater challenges to parallelization than compression, as the decoding process must reproduce the exact bit sequence in a strictly sequential manner. CuCM also achieves decompression parallelism at both the chunk and bit levels: ❶ chunk-level parallelism enabled by reusing the prelearned model, and ❷ bit-level parallelism enabled by speculation-based decompression.

Decompression in CuCM mirrors the compression process at the chunk level. It first decodes the prelearning segment to reconstruct the prelearned model, and then decompresses the remaining chunks independently using this model as the shared initialization. This design allows CuCM to fully exploit GPU parallel capacity at the chunk level while maintaining consistency with the model used during compression.

While chunk-level independence provides coarse-grained concurrency, bit-level parallelism in decompression is much more difficult due to the autoregressive modeling step of context mixing. The decoder cannot determine the actual value of the next bit until it is decoded by the entropy coder, which inherently limits straightforward parallel execution.

To overcome this limitation, CuCM employs a speculation-based decompression strategy. The key idea is to let GPU threads speculatively predict several future bits before their actual values are decoded. Each thread temporarily assumes one of the possible outcomes for multiple bits and performs the corresponding prediction and mixing operations in parallel. Once the entropy coder reveals the true bit sequence, CuCM verifies these speculative results: predictions based on correct guesses are retained and committed, while those based on incorrect guesses are discarded. This speculative execution keeps GPU cores actively utilized while the entropy coder is still processing, effectively hiding the latency of sequential bit decoding. However, this parallelism is inherently limited: to correctly decompress an n -bit sequence in advance, 2^n speculative threads are required, whereas in compression these threads can directly output the predictions of the next 2^n bits without guessing. Despite this

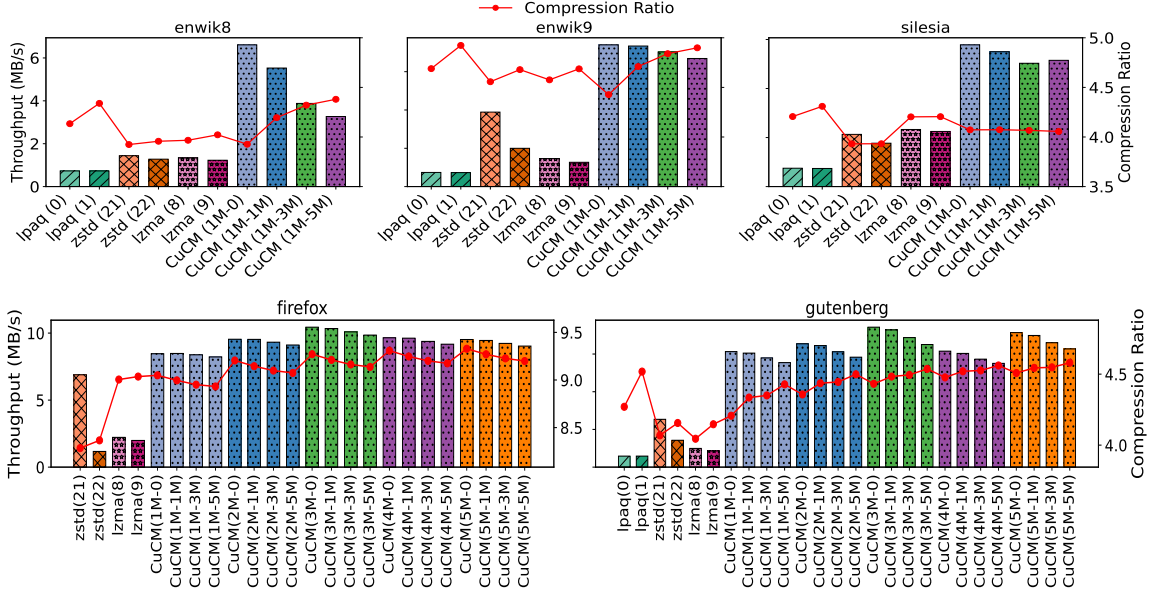


Figure 3: Compression ratio/speed of evaluated compressors. CuCM(x M- y M) denotes a chunk size of x MB with y MB prelearning. Vector size = 128. Firefox dataset is omitted for LPAQ due to its size.

limitation, speculation-based decompression significantly improves decoding efficiency without affecting correctness.

4 Experiments

4.1 Experimental Setup

Experiments run on a server with an Intel(R) Xeon(R) Platinum 8468V CPU @ 2.60GHz, 125GB RAM and an NVIDIA L40S GPU. Four approaches are evaluated:

- ❶ LZMA [6]: a widely used archival compressor;
- ❷ ZSTD with level 21/22 [7]: a popular compressor with archival configurations;
- ❸ LPAQ [9]: the fast context-mixing compressor;
- ❹ CuCM: our approach.

Approaches are evaluated on three famous general-purpose datasets (silesia [17], enwik8, enwik9 [18]) and two archival datasets: ❶ Firefox crash reporter: archived crash reporter of Firefox (2.1GB, from <https://archive.mozilla.org>), and ❷ Gutenberg: archived books from Gutenberg (854MB, from <https://www.gutenberg.org>).

4.2 General Compression Results

As shown in Fig. 3, we compare the compression ratio and throughput of CuCM with representative CPU-based compressors under different compression levels. To ensure a fair comparison, all CPU-based compressors that support multi-threading (ZSTD and LZMA) were executed using 16 threads, which already saturates the compression throughput on our test CPU.

Compression Speed. Overall, CuCM achieves substantial throughput improvements, reaching up to $12.6\times$ higher compression speed than LPAQ while maintaining competitive compression ratios.

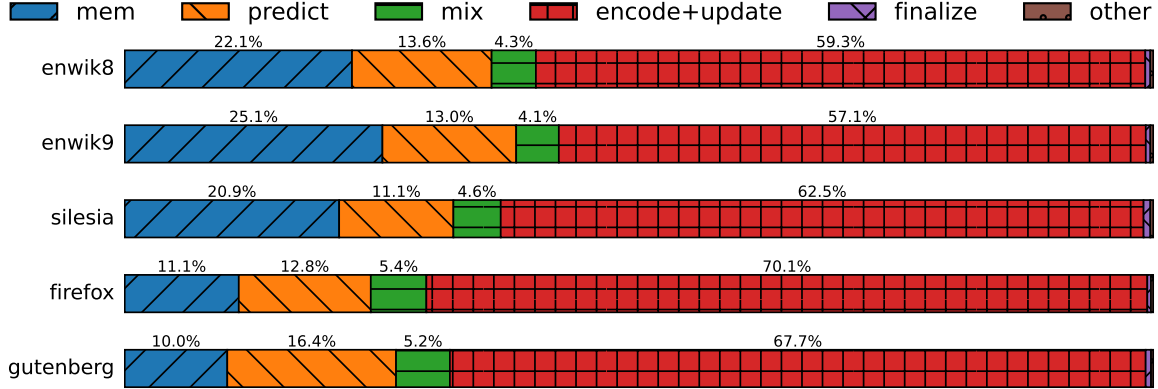


Figure 4: The time breakdown of CuCM during compression on evaluated datasets.

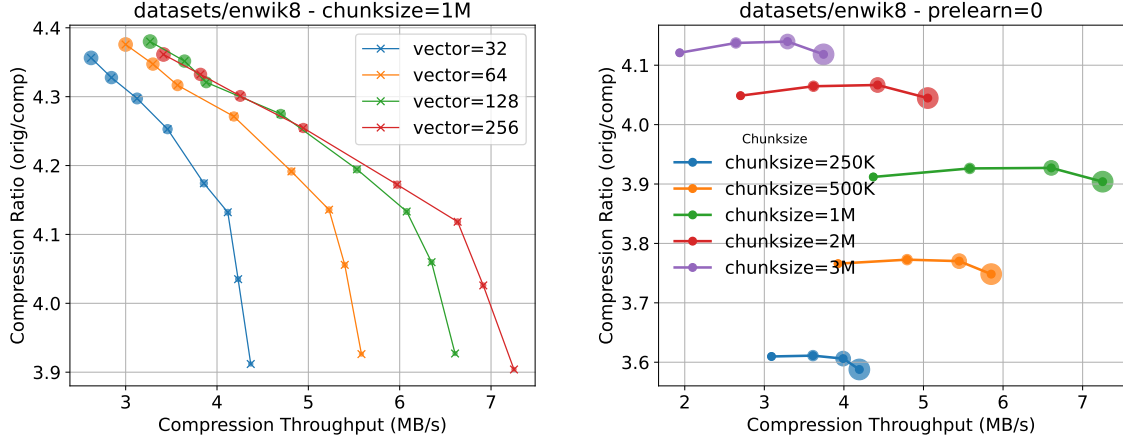
Compression Ratio and the Effectiveness of Prelearning. CuCM delivers strong performance even without dataset-specific tuning. The baseline configuration (CuCM-1M-0, without prelearning) already achieves comparable ratios to state-of-the-art methods, while enabling prelearning (CuCM-1M-1M/3M/5M) further improves compression effectiveness. This benefit is particularly evident on datasets with consistent statistical characteristics, such as enwik8 and Gutenberg, where the learned model generalizes well across the entire file.

The Impact of Chunk Size. We also observe that larger chunk sizes tend to enhance compression ratio at the cost of lower throughput, as illustrated by the results on the Firefox and Gutenberg datasets. But this tradeoff is also influenced by both the total number of chunks and the GPU memory constraint. For example, increasing the chunk size from 1MB to 3MB in the Firefox and Gutenberg datasets improves compression ratio without affecting the degree of GPU parallelism, since the number of concurrently processed chunks remains similar. In this case, fewer processing rounds are required, leading to overall better efficiency.

Compression Time Breakdown of CuCM. Fig. 4 presents the time breakdown of CuCM across different operations during compression. “Mem” means the time used in GPU memory allocation and data transfer between GPU and CPU; “predict” and “mix” are the time used in the prediction and mixing kernels; “encode” and “update” kernels are launched together so their time is counted together; “finalize” means the time used in kernels calculating the offsets of each chunk and flushing. “Encode+update” dominate the overall runtime, as they are the most computationally intensive stages of CuCM and involve extensive random memory accesses. The overhead from “mem” is also non-negligible, indicating that further optimization of memory layout and transfer scheduling could improve overall throughput.

4.3 Influence of Vector Size

Vector-based coding is a key technique in CuCM. To understand its effect, we conduct detailed experiments to examine how different vector sizes influence both compression ratio and throughput. As shown in Fig. 5, increasing the vector size consistently improves throughput under all configurations, which aligns with our expectation that longer vectors allow more bits to be processed in parallel.



(a) Effect of vector size and prelearning size on CuCM performance. Each line represents a fixed vector size. Circle sizes indicate different prelearning size (0–5 MB).

(b) Effect of vector size and chunk size on CuCM performance. Each line represents a fixed chunk size. Circle sizes indicate different vector size (32–256 bits).

Figure 5: The influence of vectorsize on compression ratio and throughput.

Interestingly, enlarging the vector size has only a limited impact on compression ratio. In most cases, a vector size of 128 achieves the best ratio, slightly outperforming smaller settings (64 or 32), while further increasing it to 256 results in a noticeable degradation. This trend can be explained by the design of vector-based prediction: bits within a vector are predicted independently, without inter-bit interaction. A moderate vector size (e.g., 128) helps the model smooth out short-term noise caused by hash collisions and better capture longer-range dependencies in the input. However, when the vector becomes too large, these dependencies may be diluted, leading to a loss in compression efficiency.

4.4 The effectiveness of speculation prediction in decompression

We further evaluate the effectiveness of speculative prediction during decompression. Although the detailed breakdown is omitted here due to space constraints, the results show that enabling speculation substantially improves throughput, achieving up to an $8.75\times$ speedup compared to the CuCM without speculation. A speculation size of 9 provides the best balance between prediction accuracy and execution efficiency, while larger speculation windows offer diminishing returns because of increased misprediction overhead.

4.5 Discussion

As shown in Fig.4, model updating is the most time-consuming step during compression. Our current implementation introduces a prelearning stage at the a small part of the input stream, enabling CuCM to operate without any prior knowledge of the data. However, if a well-trained model were reused for datasets sharing similar statistical distributions or data semantics with the prelearned corpus, CuCM could bypass runtime model updates and thereby achieve high compression efficiency and throughput at the same time. For example, when CuCM keeps its model fixed after

prelearning on 10MB of enwik8, it achieves a compression ratio of 3.59 and a throughput of 191 MB/s on the remaining data, already superior to other approaches. This suggests CuCM’s potential to be substantially accelerated if a well-initialized model were available.

5 Conclusion

We present CuCM, a GPU-powered context-mixing compressor. CuCM introduces prelearning and batch modeling techniques to break the sequential dependencies in previous context-mixing compressor, enabling a high degree of parallelism that fully leverages the massive arithmetic throughput and parallelism of modern GPUs. Evaluation results show that CuCM achieves up to $12.6\times$ higher compression throughput than existing archival compressors while maintaining comparable compression ratios.

References

- [1] A. Wright, “Worldwide idc global datasphere forecast, 2024–2028: Ai everywhere, but upsurge in data will take time,” 2024.
- [2] D. Monroe, “Addressing the data storage crisis,” *Commun. ACM*, vol. 68, no. 1, pp. 17–19, 2025.
- [3] Y. Fu, J. Su, J. Ning, J. Wu, Y. Lu, and N. Xiao, “Distributed data deduplication for big data: A survey,” *ACM Computing Surveys*, vol. 58, no. 3, 2025.
- [4] Y. Gao and A. G. Parameswaran, “Squish: Near-optimal compression for archival of relational datasets,” in *Proceedings of the 22nd International Conference on Knowledge Discovery and Data Mining (ACM SIGKDD)*, 2016, pp. 1575–1584.
- [5] G. Pekhimenko, C. Guo, M. Jeon, P. Huang, and L. Zhou, “Tersecades: Efficient data compression in stream processing,” in *USENIX Annual Technical Conference (USENIX ATC)*, 2018, pp. 307–320.
- [6] I. Pavlov, “lzma sdk,” <https://www.7-zip.org/sdk.html>, 2001.
- [7] Y. Collet, “zstd,” <https://facebook.github.io/zstd>, 2016.
- [8] J. Ziv and A. Lempel, “A universal algorithm for sequential data compression,” *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977.
- [9] M. Mahoney, “lpaq,” <http://mattmahoney.net/dc/#lpaq>, 2007.
- [10] B. Knoll, “cmix,” <https://www.byronknoll.com/cmix.html>, 2014.
- [11] F. Bellard, “nnpc,” <https://bellard.org/nnpc>, 2006.
- [12] M. Mahoney, “Large text compression benchmark (ltcb),” <https://www.mattmahoney.net/dc/text.html>, 2006.
- [13] W. Zhu, W. Tong, H. Ge, Z. Zhang, M. Zhang, and W. Zhou, “Lpaqhp: A high-performance fpga accelerator for lpaq compression,” in *Proceedings of the 53rd International Conference on Parallel Processing (ICPP)*, 2024, p. 898–907.
- [14] D. Tang, X. Sun, N. Guan, T.-W. Kuo, and C. J. Xue, “plpaq: Accelerating lpaq compression on fpga,” in *Proceedings of 2022 International Conference on Field-Programmable Technology (ICFPT)*, 2022, pp. 1–6.
- [15] NVIDIA, “Nvidia l40s datasheet,” 2025.
- [16] Intel, “App metrics for intel® microprocessors,” 2025.
- [17] S. Deorowicz, “The silesia corpus,” <http://www.data-compression.info/Corpora/SilesiaCorpus/index.html>, 2003.
- [18] M. Mahoney, “enwik8/enwik9 dataset,” <http://mattmahoney.net/dc/textdata.html>, 2006.